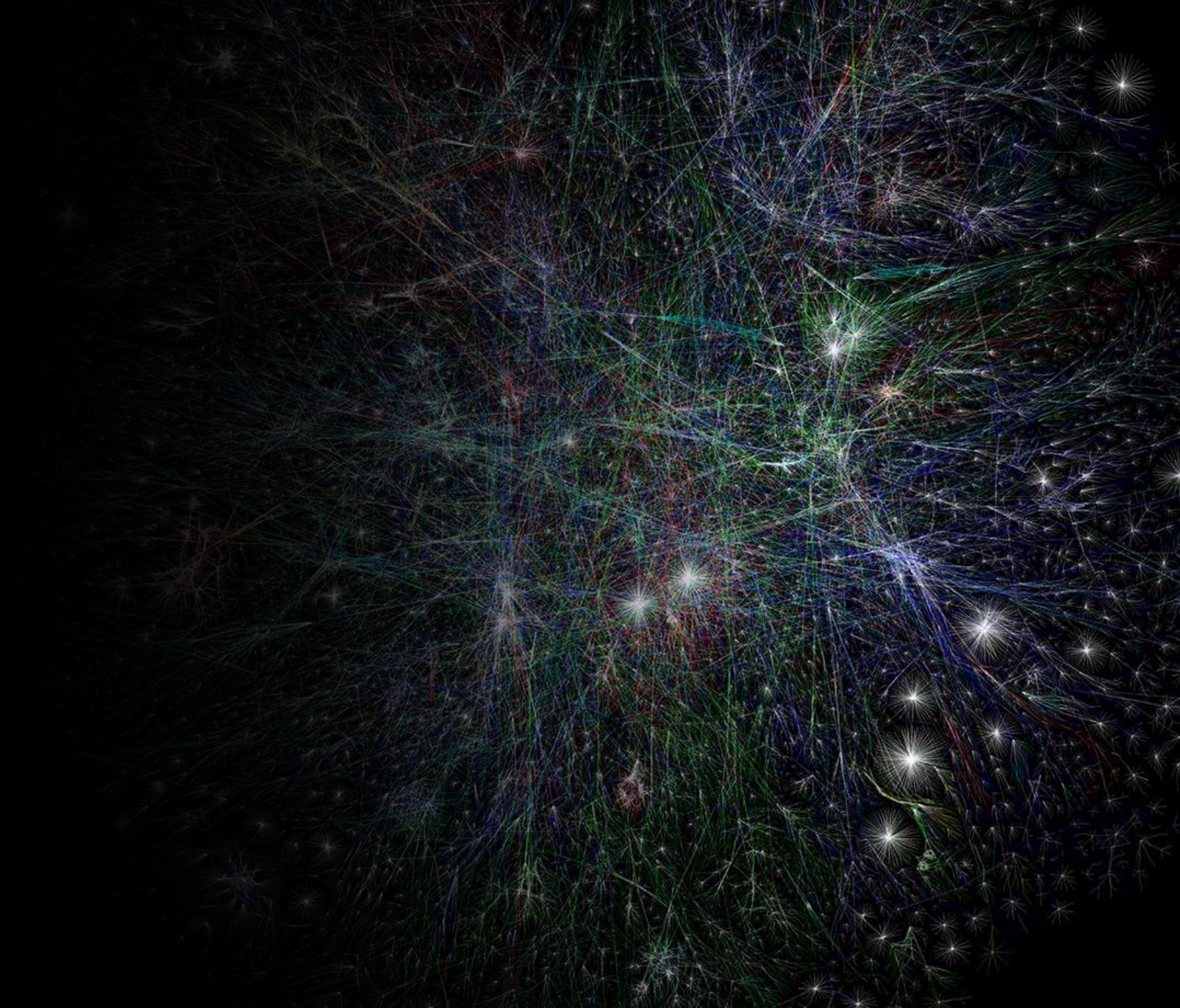
INFO 215: Web Science.
Lecture 11: Network modularity

27 March, 2025

- *Fazle Rabbi, Ph.D*
- *Assoc. Prof in Information Science*
- *University of Bergen*
- *Fazle.Rabbi@uib.no*
- *Bergen, Norway*



Learning Objectives:

Theoretical knowledge:
Community detection algorithm
Graph partitioning

Practical knowledge:
Hands on experience on finding communities
Python library for community detection/clustering

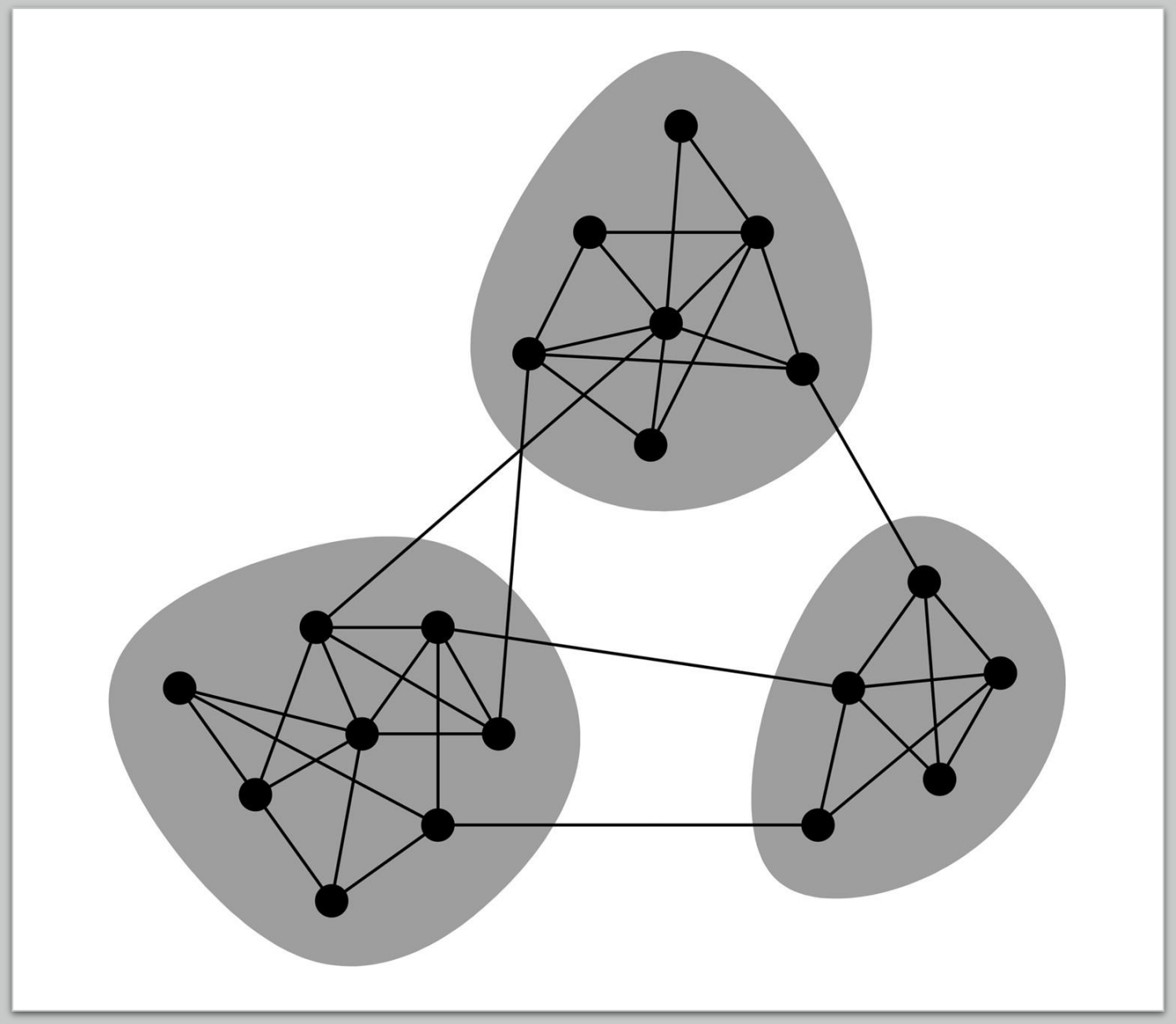
Community

What is a community?

- Nodes form tightly knit groups called communities.

How to identify communities?

- Number of edges that connect vertices within a community are high.
- Number of edges across communities are low.



Community detection algorithm

The algorithms for community detection are categorized into approaches based on

- graph partitioning,
- clustering,
- Louvain method
- label propagation

Graph partitioning

Denote by C = $\{C_1, C_2,\ldots,C_k\}$ a partition of the vertices in V. Each C_i is called community. Each community C_i is associated to an induced sub-graph

$G(C_i) = (V_{Ci}, E_{Ci})$ of G, where V_{Ci} is a subset of V, and $E_{Ci} = \{(u, v): u, v \in V_{Ci}, (u, v) \in E\}$.

Graph partitioning

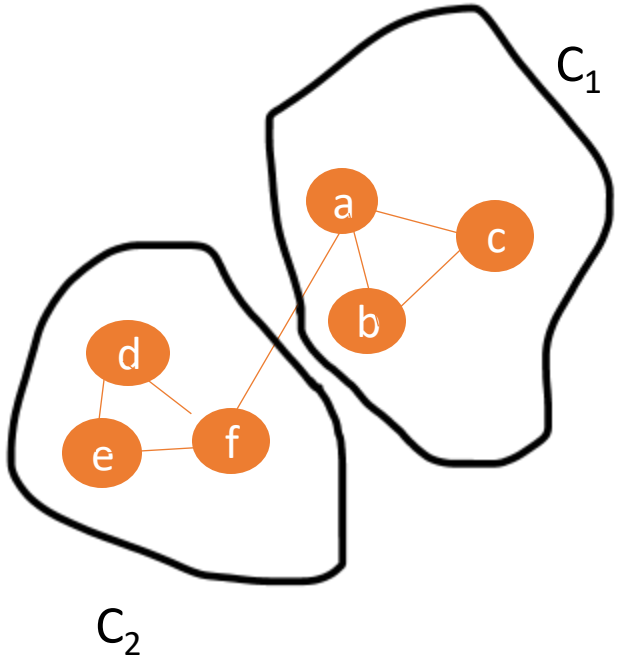
Denote by C = {C_1, C_2,...,C_k} a partition of the vertices in V. Each C_i is called community. Each community C_i is associated to an induced sub-graph

$G(C_i) = (V_{Ci}, E_{Ci})$ of G, where V_{Ci} is a subset of V, and $E_{Ci} = \{(u, v): u, v \in V_{Ci}, (u, v) \in E\}$.

Example:
G = (V, E)
V = {a,b,c,d,e,f}
E = { (a,b), (a,c), (b,c), (a,f), (d,e), (d,f), (e,f)}

C = {C_1, C_2}
$G(C_1) = (V_{C1}, E_{C1})$; $G(C_2) = (V_{C2}, E_{C2})$
V_{C1} = {a,b,c}
E_{C1} = {(a,b), (a,c), (b,c)}

V_{C2} = {d,e,f}
E_{C2} = {(d,e), (d,f), (e,f)}



Graph partitioning

Denote by C = $\{C_1, C_2,...,C_k\}$ a partition of the vertices in V. Each C_i is called community. Each community C_i is associated to an induced sub-graph

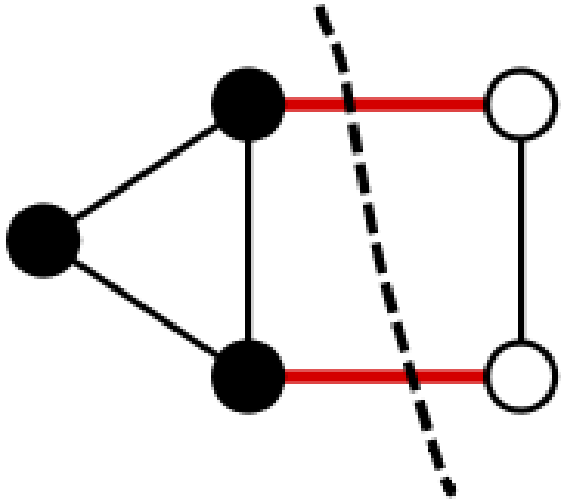
$G(C_i) = (V_{Ci}, E_{Ci})$ of G, where V_{Ci} is a subset of V, and $E_{Ci} = \{(u, v): u, v \in V_{Ci}, (u, v) \in E\}$.

A good community detection strategy should strive to achieve two conflicting goals:

- provide an accurate network partition
- keep low the computational complexity of the algorithm so as the algorithm can be applied to very large networks.

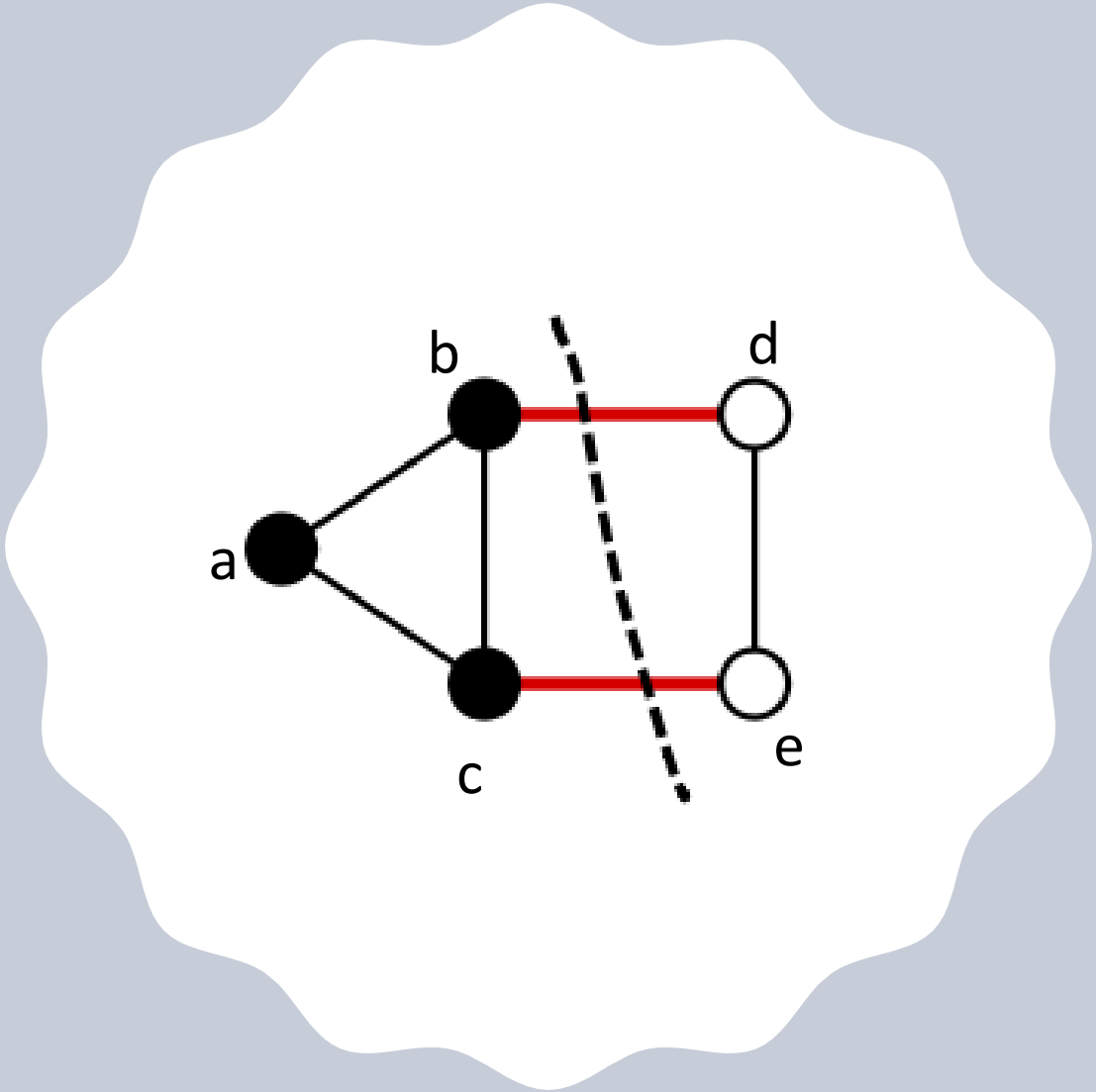
Graph cut

- Sets of edges whose removal makes a graph disconnected.
- A **cut** C=(S,T) is a partition of V of a graph G=(V,E) into two subsets *S* and *T*. The **cut-set** of a cut C=(S,T) is the set {(u,v) ∈ E | u ∈ S, v ∈ T } of edges that have one endpoint in *S* and the other endpoint in *T*. If *s* and *t* are specified vertices of the graph *G*, then an ***s–t* cut** is a cut in which *s* belongs to the set *S* and *t* belongs to the set *T*.
- Cost of a cut is determined by the sum of weights of cut edges.



Graph cut

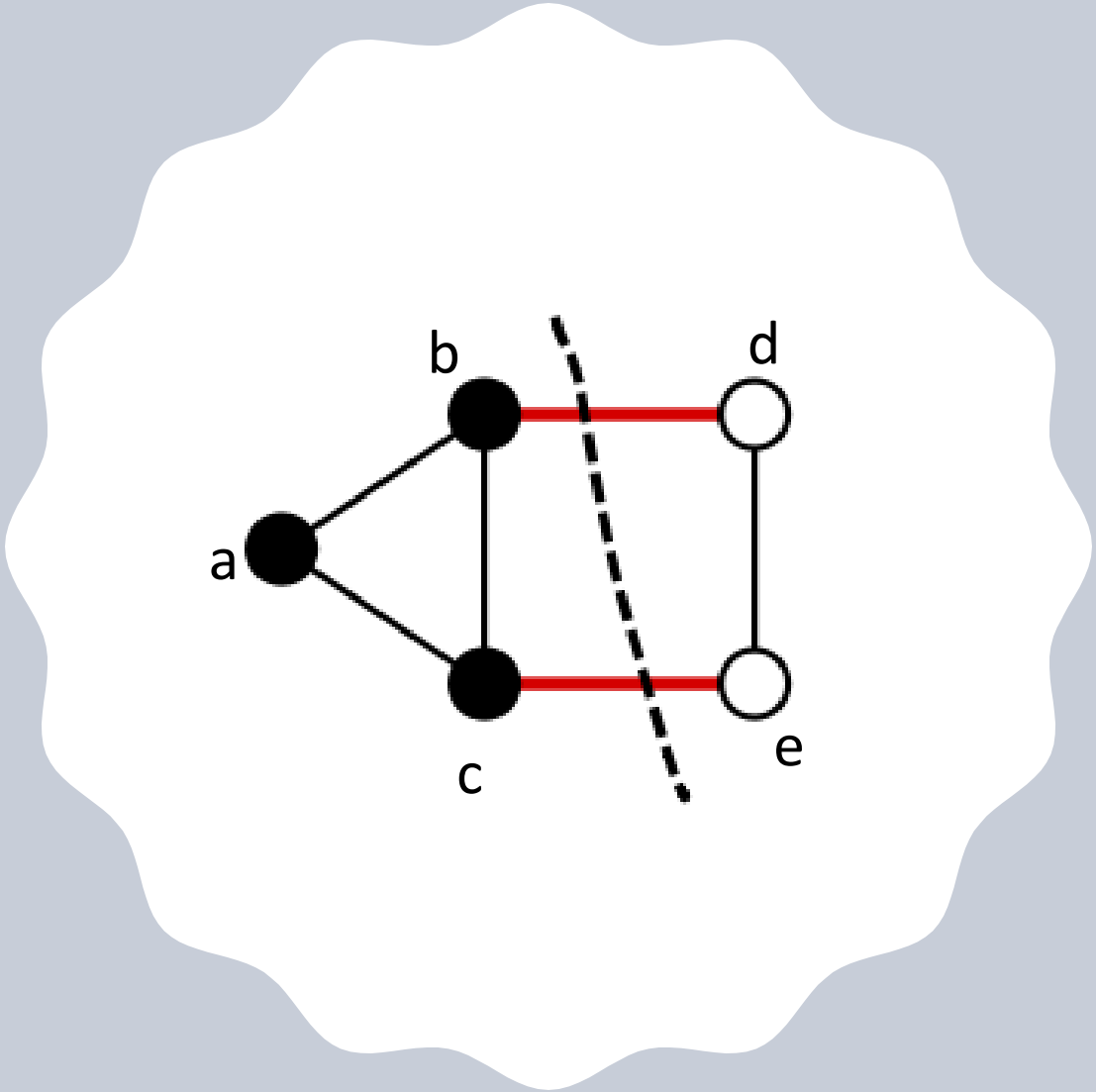
- A **cut** C=(S,T) is a partition of V of a graph G=(V,E) into two subsets *S* and *T*. The **cut-set** of a cut C=(S,T) is the set {(u,v) ∈ E | u ∈ S, v ∈ T } of edges that have one endpoint in *S* and the other endpoint in *T*.



- Example:
- G = (V, E)
- V = {a,b,c,d,e}
- E = {(a,b), (a,c), (b,c), (b,d), (c,e), (d,e)}
- Cut C = (S, T)
- S = {a,b,c}
- T = {d,e}
- What is the cut-set of cut C?
- cut-set of cut C is { (b,d), (c,e)}

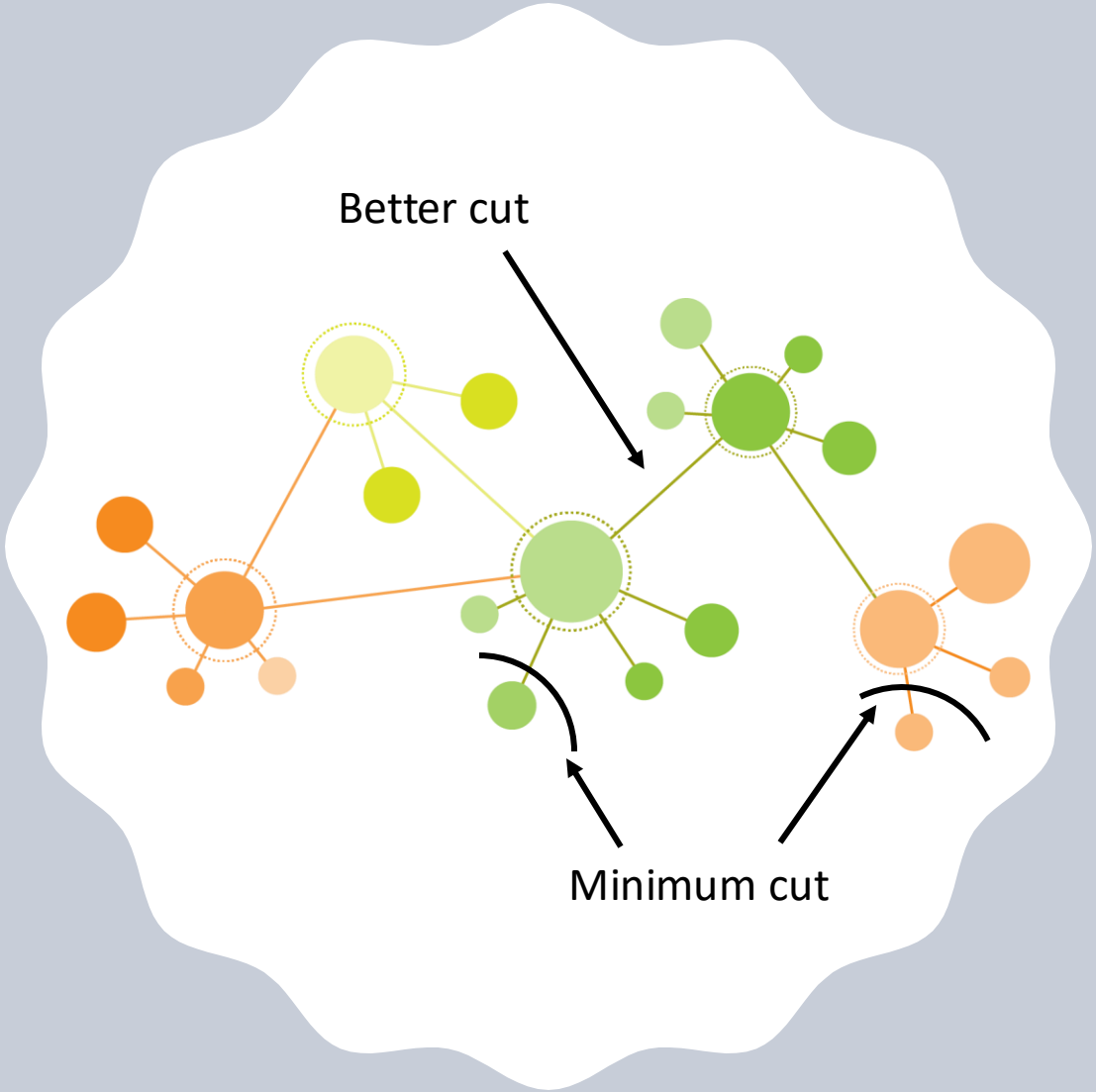
Quality of graph cut: Minimum cut

- A cut is *minimum* if the size or weight of the cut is not larger than the size of any other cut. The illustration on the right shows a minimum cut: the size of this cut is 2, and there is no cut of size 1 because the graph is bridgeless.



Quality of graph cut: Minimum cut

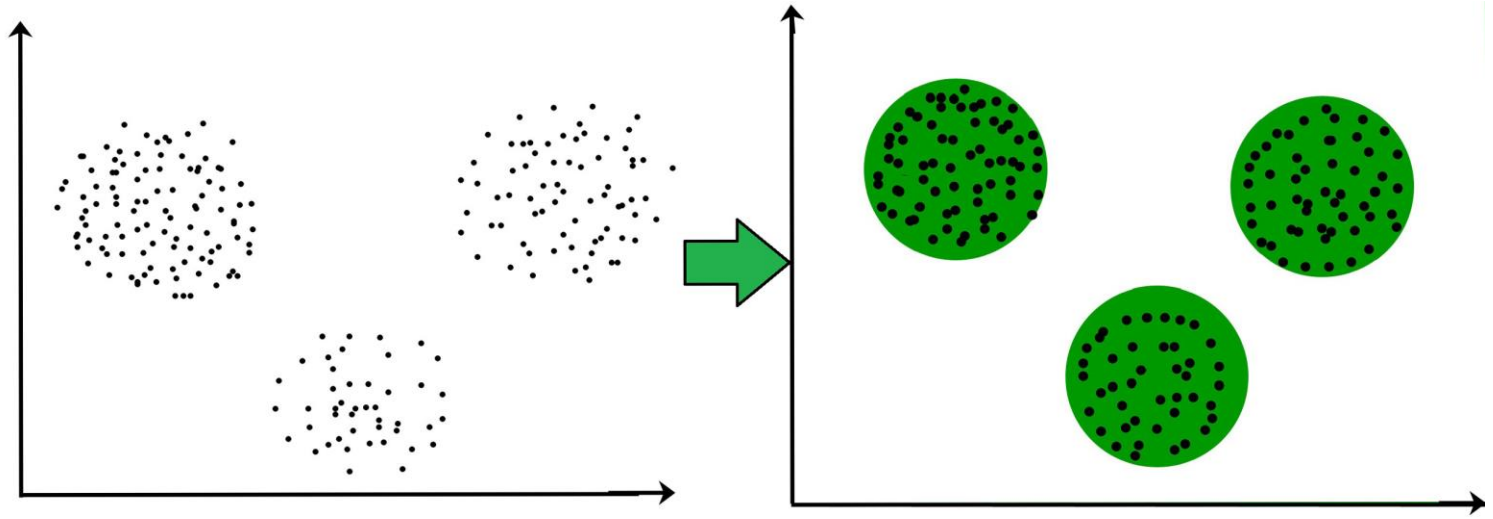
- A cut is *minimum* if the size or weight of the cut is not larger than the size of any other cut.
- Problems with minimum cut:
  - Tends to produce small isolated components.



Clustering

Clustering divides the population or data points into a number of groups such that:

- data points in the same groups are more similar to other data points in the same group,
- data points from dissimilar groups have less similarity.

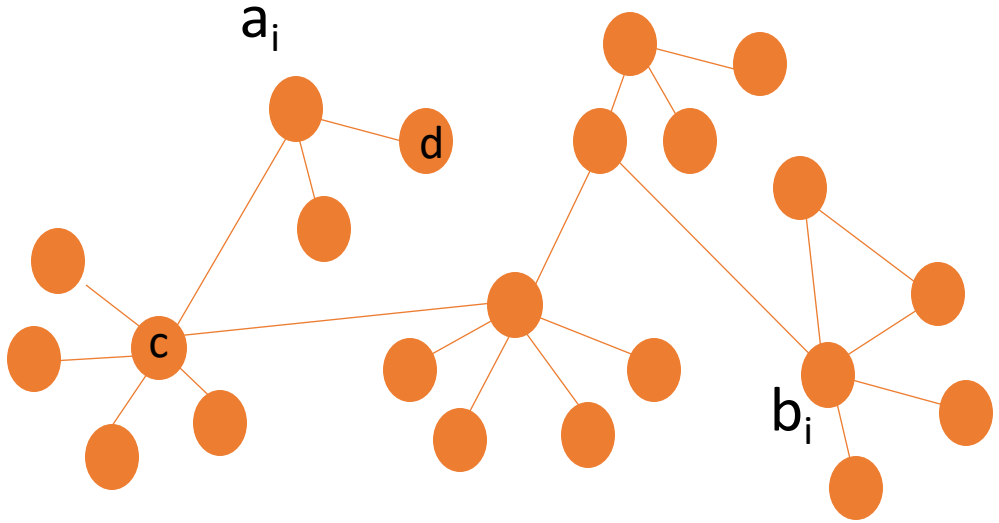


K means clustering

It is an unsupervised machine learning algorithm that solves clustering problem.

K-means algorithm partition n observations into k clusters where each observation belongs to the cluster with the nearest mean serving as a prototype of the cluster .

K means clustering



When k=2, we want to identify 2 clusters.

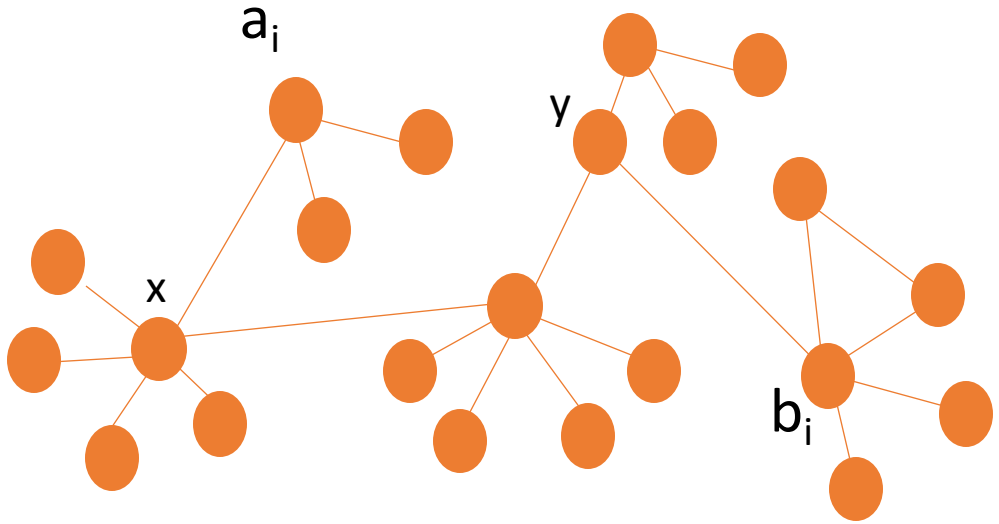
Problem: Find x, y (center points) to minimize the following:

$$\Sigma \text{ dist}(a_i , x) + \Sigma \text{ dist}(b_i , y)$$

Where set of all a's U set of all b's = all the vertices

And the intersection of a's and b's are empty.

K means clustering



Steps for computing the centroids:

1. Given a's and b's find centroids.
2. From a given x and y, form clusters
3. Restart step 1 until the algorithm converges.

Modularity

- The modularity of a partition is a scalar value between -1 and 1 that measures the density of links inside communities as compared to links between communities.
- In the case of weighted networks, it is defined as:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j), \tag{1}$$

where A_{ij} represents the weight of the edge between i and j, $k_i = \sum_j A_{ij}$ is the sum of the weights of the edges attached to vertex i, c_i is the community to which vertex i is assigned, the δ-function $\delta(u, v)$ is 1 if $u = v$ and 0 otherwise and $m = \frac{1}{2} \sum_{ij} A_{ij}$.

[Vincent et. al: Fast unfolding of communities in large networks, 2018]

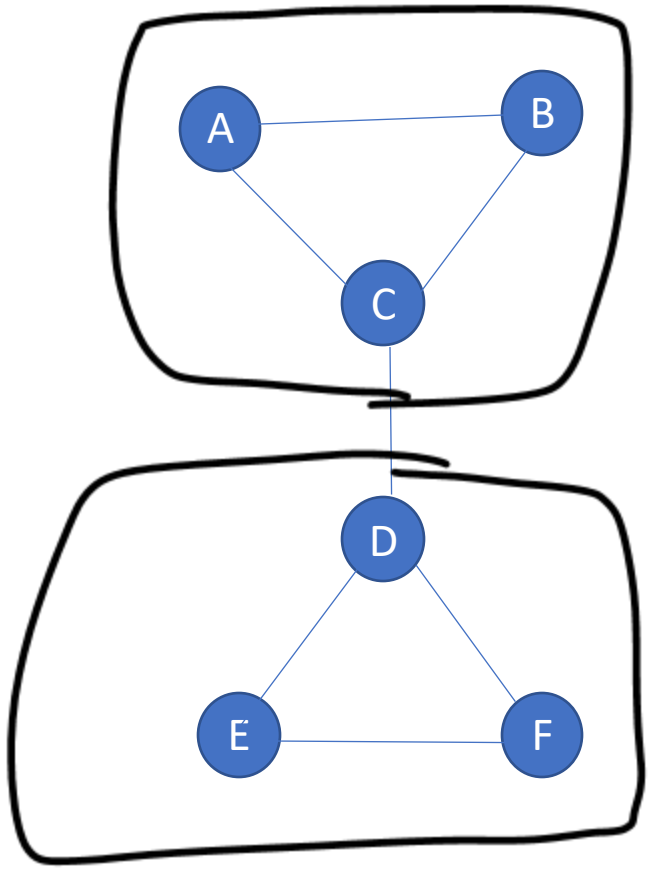
Modularity

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j), \tag{1}$$

where A_{ij} represents the weight of the edge between i and j, $k_i = \sum_j A_{ij}$ is the sum of the weights of the edges attached to vertex i, c_i is the community to which vertex i is assigned, the δ-function $\delta(u, v)$ is 1 if $u = v$ and 0 otherwise and $m = \frac{1}{2} \sum_{ij} A_{ij}$.

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	1	0	0	0
C	1	1	0	1	0	0
D	0	0	1	0	1	1
E	0	0	0	1	0	1
F	0	0	0	1	1	0

m = 7



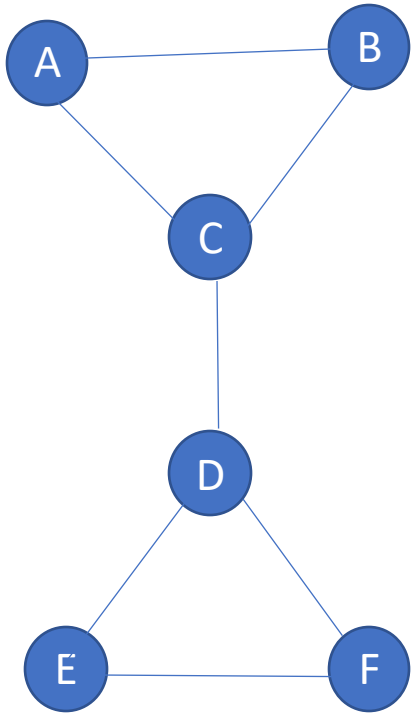
Modularity

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j), \qquad (1)$$

where A_{ij} represents the weight of the edge between i and j, $k_i = \sum_j A_{ij}$ is the sum of the weights of the edges attached to vertex i, c_i is the community to which vertex i is assigned, the δ-function $\delta(u, v)$ is 1 if $u = v$ and 0 otherwise and $m = \frac{1}{2} \sum_{ij} A_{ij}$.

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	1	0	0	0
C	1	1	0	1	0	0
D	0	0	1	0	1	1
E	0	0	0	1	0	1
F	0	0	0	1	1	0

m = 7
k_A = 2
k_B = 2
k_C = 3
k_D = 3
k_E = 2
k_F = 2



Here (k_i k_j / 2m) is an approximate expression which represents the expected number of edges between two nodes.

Modularity

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j), \qquad (1)$$

where A_{ij} represents the weight of the edge between i and j, $k_i = \sum_j A_{ij}$ is the sum of the weights of the edges attached to vertex i, c_i is the community to which vertex i is assigned, the δ-function $\delta(u, v)$ is 1 if $u = v$ and 0 otherwise and $m = \frac{1}{2} \sum_{ij} A_{ij}$.

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	1	0	0	0
C	1	1	0	1	0	0
D	0	0	1	0	1	1
E	0	0	0	1	0	1
F	0	0	0	1	1	0

m = 7

$k_A = 2$

$k_B = 2$

$k_C = 3$

$k_D = 3$

$k_E = 2$

$k_F = 2$

$c_A = c_1$

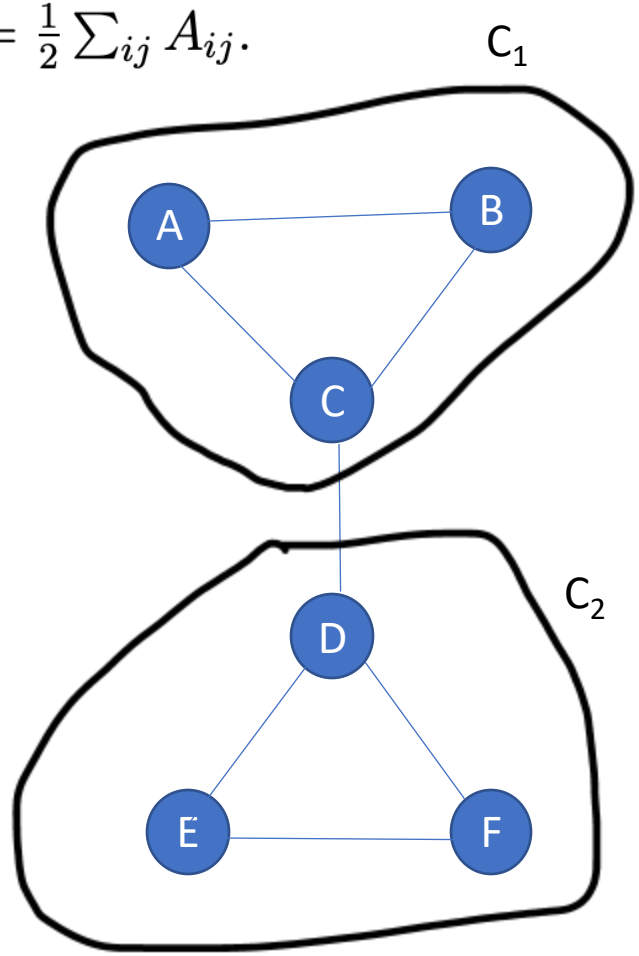
$c_B = c_1$

$c_C = c_1$

$c_D = c_2$

$c_E = c_2$

$c_F = c_2$



Modularity

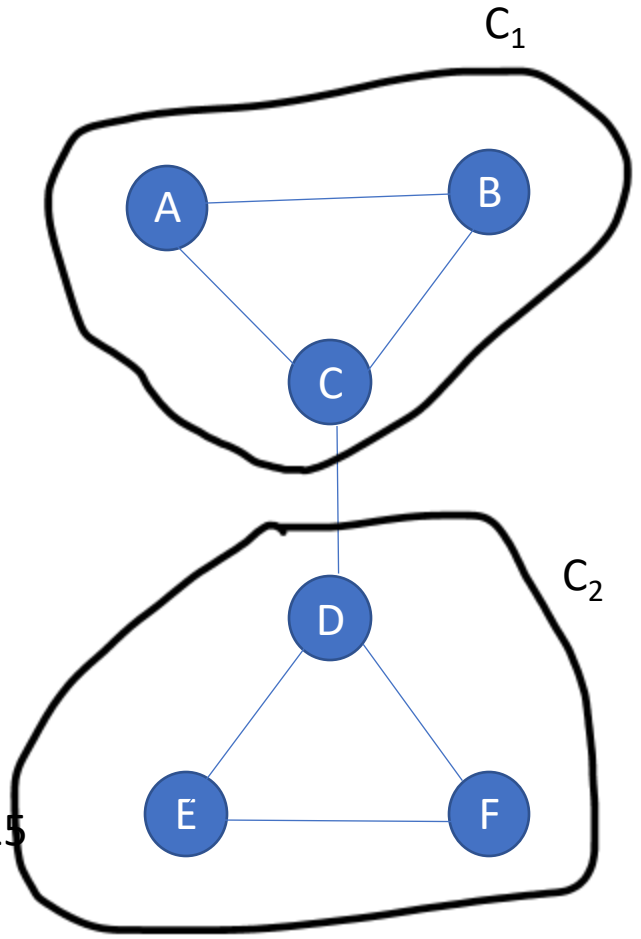
$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j), \qquad (1)$$

where A_{ij} represents the weight of the edge between i and j, $k_i = \sum_j A_{ij}$ is the sum of the weights of the edges attached to vertex i, c_i is the community to which vertex i is assigned, the δ-function $\delta(u, v)$ is 1 if $u = v$ and 0 otherwise and $m = \frac{1}{2} \sum_{ij} A_{ij}$.

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	1	0	0	0
C	1	1	0	1	0	0
D	0	0	1	0	1	1
E	0	0	10	1	0	1
F	0	0	0	1	1	0

m = 7
$k_A = 2$
$k_B = 2$
$k_C = 3$
$k_D = 3$
$k_E = 2$
$k_F = 2$

$c_A = c_1$
$c_B = c_1$
$c_C = c_1$
$c_D = c_2$
$c_E = c_2$
$c_F = c_2$



$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j),$$

when i = A and j = B, then

$$[A_{AB} - \frac{k_A k_B}{14}] \quad \delta(c_A, c_B) = [1 - \frac{2 \times 2}{14}] \times 1 = 0.715$$

Modularity

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	1	0	0	0
C	1	1	0	1	0	0
D	0	0	1	0	1	1
E	0	0	0	1	0	1
F	0	0	0	1	1	0

$m = 7$

$k_A = 2$

$k_B = 2$

$k_C = 3$

$k_D = 3$

$k_E = 2$

$k_F = 2$

$c_A = c_1$

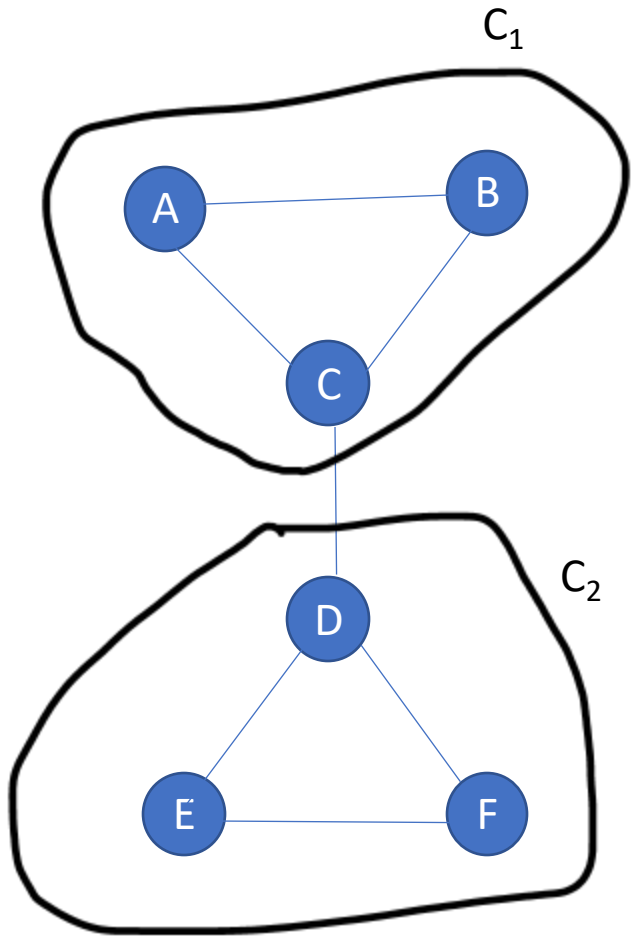
$c_B = c_1$

$c_C = c_1$

$c_D = c_2$

$c_E = c_2$

$c_F = c_2$



$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j),$$

when i = A and j = B, then

$$[A_{AB} - \frac{k_A k_B}{14}] \, \delta(c_A, c_B) = [1 - \frac{2 \times 2}{14}] \times 1 = 0.715$$

when i = A and j = C, then

$$[A_{AC} - \frac{k_A k_C}{14}] \, \delta(c_A, c_C) = [1 - \frac{2 \times 3}{14}] \times 1 = 0.58$$

when i = C and j = D, then

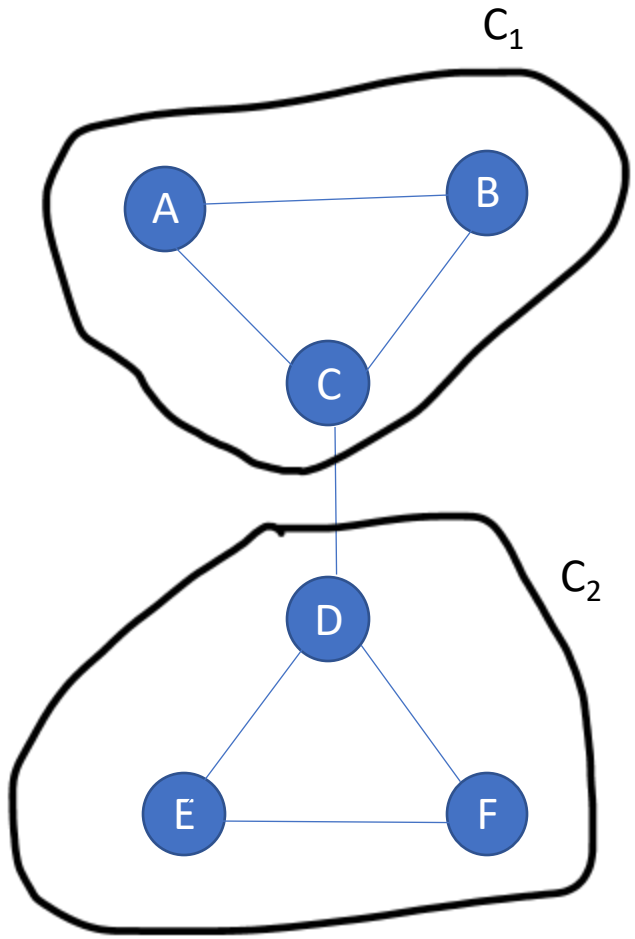
$$[A_{CD} - \frac{k_C k_D}{14}] \, \delta(c_C, c_D) = [1 - \frac{3 \times 3}{14}] \times 0 = 0$$

Modularity

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	1	0	0	0
C	1	1	0	1	0	0
D	0	0	1	0	1	1
E	0	0	0	1	0	1
F	0	0	0	1	1	0

m = 7
$k_A = 2$
$k_B = 2$
$k_C = 3$
$k_D = 3$
$k_E = 2$
$k_F = 2$

$c_A = c_1$
$c_B = c_1$
$c_C = c_1$
$c_D = c_2$
$c_E = c_2$
$c_F = c_2$



$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j),$$

$$Q = \frac{1}{14} [0.715 + 0.58 + 0.58 + 0 + 0.58 + 0.58 + 0.715]$$

Note: AB. AC. BC. CD. DE. DF. EF.

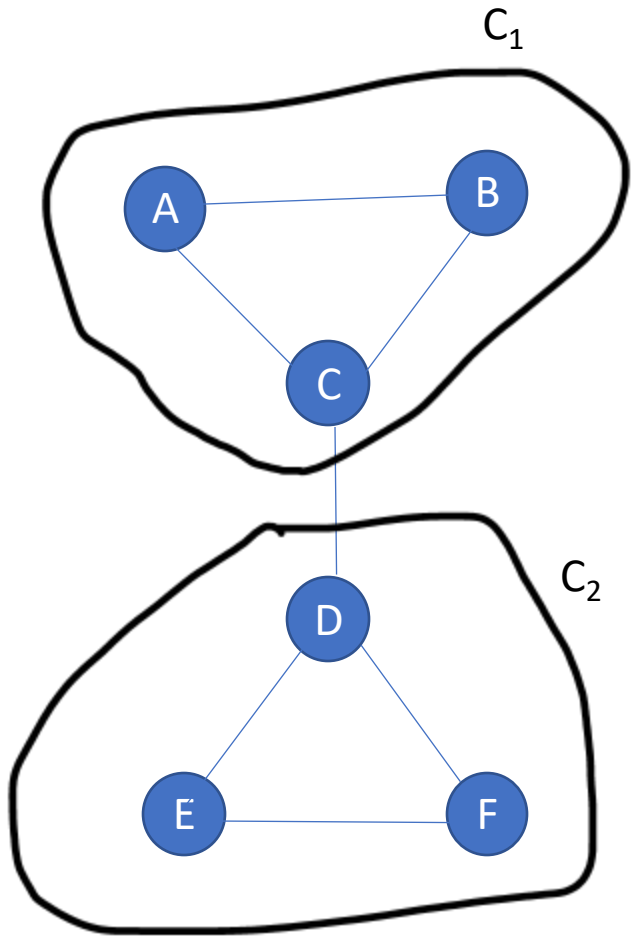
What can we conclude?
Higher modularity => more internal edges inside cluster ?

Modularity

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	1	0	0	0
C	1	1	0	1	0	0
D	0	0	1	0	1	1
E	0	0	0	1	0	1
F	0	0	0	1	1	0

m = 7
$k_A = 2$
$k_B = 2$
$k_C = 3$
$k_D = 3$
$k_E = 2$
$k_F = 2$

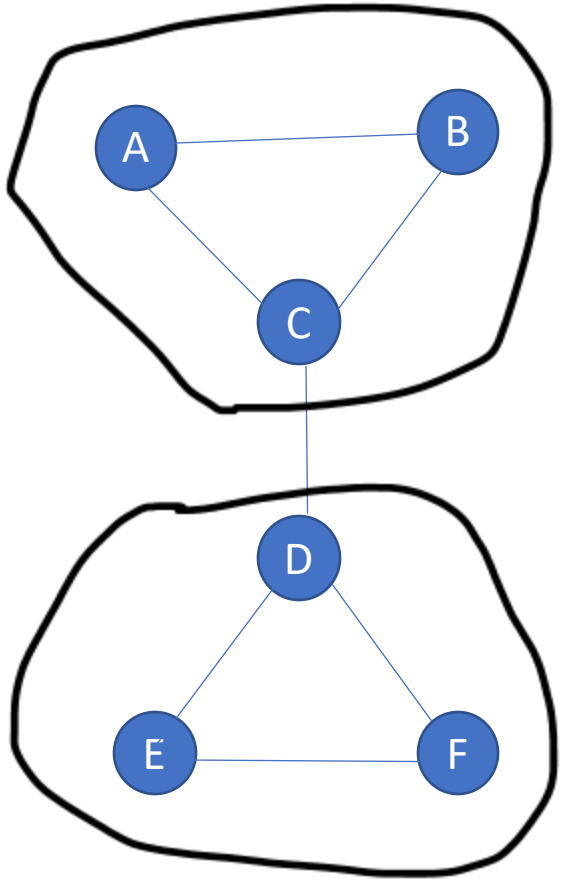
$c_A = c_1$
$c_B = c_1$
$c_C = c_1$
$c_D = c_2$
$c_E = c_2$
$c_F = c_2$



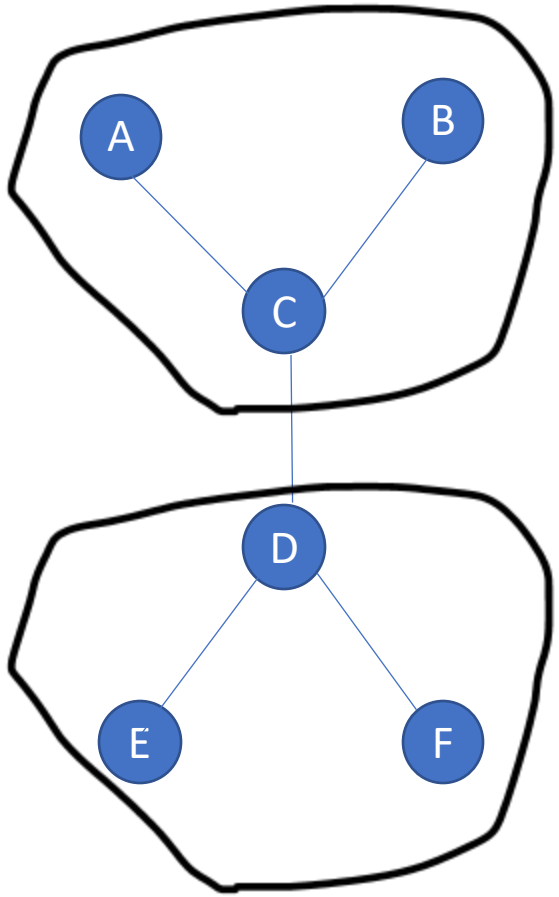
$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j),$$

Q = 1/14 [2 x 0.715 + 4 x 0.58]. = (1.43 + 2.32) / 14 = 0.26

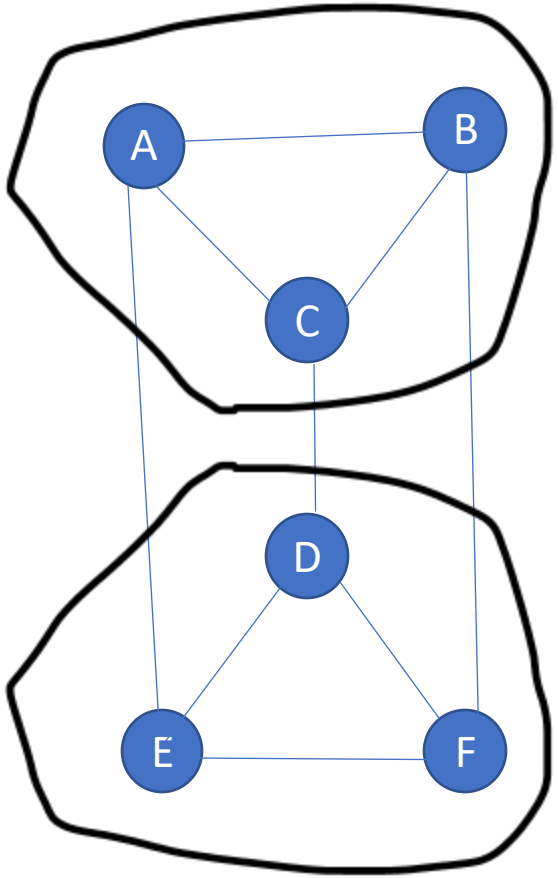
Modularity Exercise: Which graph community has the highest modularity ?



Option 1



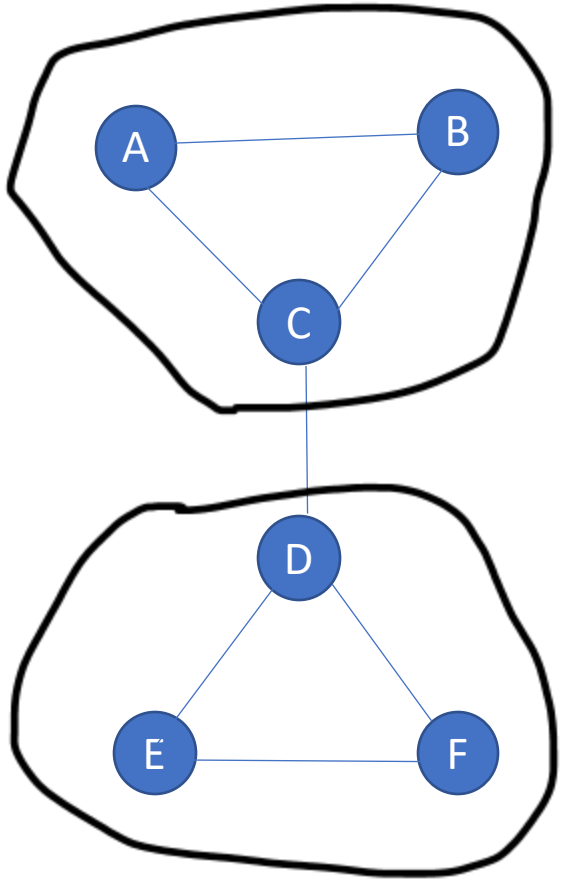
Option 2



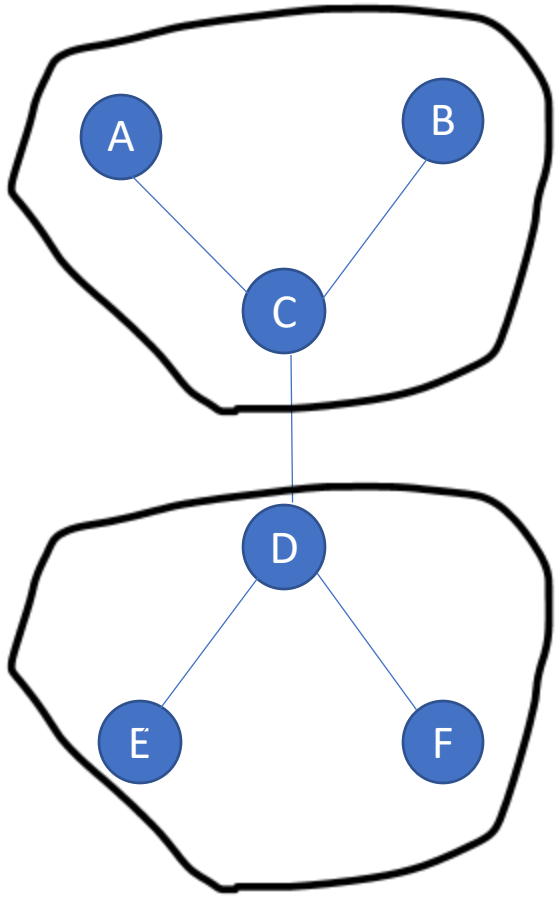
Option 3

Modularity Exercise: Which graph community has the highest modularity ?

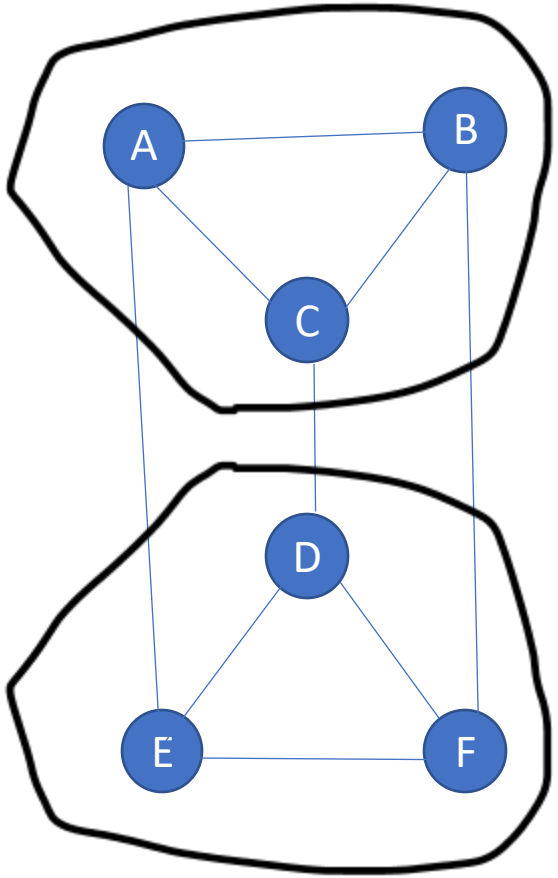
$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j),$$



Option 1



Option 2



Option 3

Modularity

- How to calculate modularity with networkx?

modularity(*G*, *communities*, *weight='weight'*)

Returns the modularity of the given partition of the graph.

Parameters

- **G** (*NetworkX Graph*)
- **communities** (*list or iterable of set of nodes*) – These node sets must represent a partition of G's nodes.
- **weight** (*string or None, optional (default="weight")*) – The edge attribute that holds the numerical value used as a weight. If None or an edge does not have that attribute, then that edge has weight 1.

Return type. float

[https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.community.quality.modularity.html]

Modularity (Example)

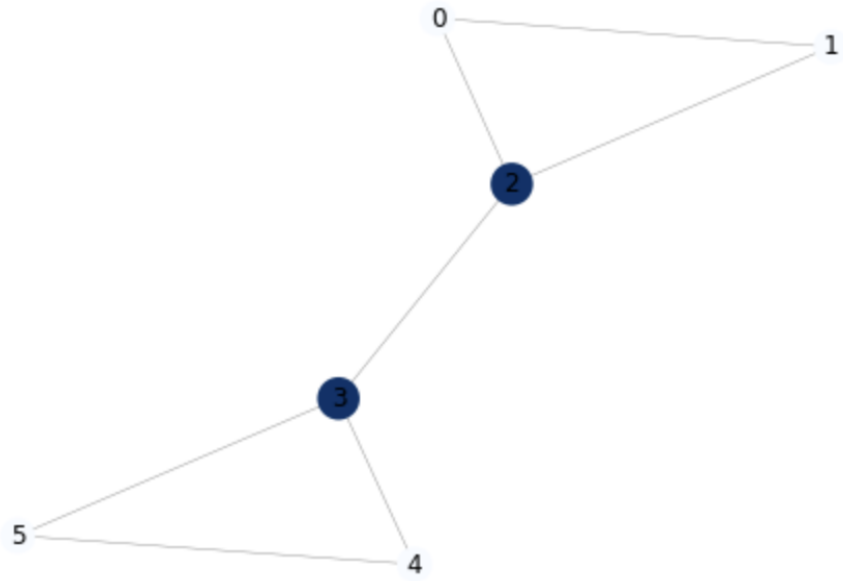
- How to calculate modularity with networkx?

import networkx.algorithms.community as nx_comm

G = nx.barbell_graph(3, 0)

nx_comm.modularity(G, [{0, 1, 2}, {3, 4, 5}])

0.35714285714285715



Todo: Try different parameters for the modularity function e.g., [{0,1},{2,3,4,5}]
Do we get maximum value for modularity when we use [{0, 1, 2}, {3, 4, 5}] ?

[https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.community.quality.modularity.html
https://www.geeksforgeeks.org/barbell-graph-using-python-networkx/]

Clauset-Newman-Moore greedy modularity maximization

- In this approach modularity begins with each node in its own community.
- The approach joins the pair of communities that most increases modularity until no such pair exists.

Networkx function to find communities based on greedy modularity maximization:

greedy_modularity_communities(*G, weight=None*)

Example:

```
from networkx.algorithms.community import
greedy_modularity_communities
G = nx.karate_club_graph()
c = list(greedy_modularity_communities(G))
sorted(c[0])
[8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33]
```

Louvain method

- Louvain method is used to extract the community structure of large networks.
- It is a heuristic method that is based on modularity optimization.
- It determines the number of communities.

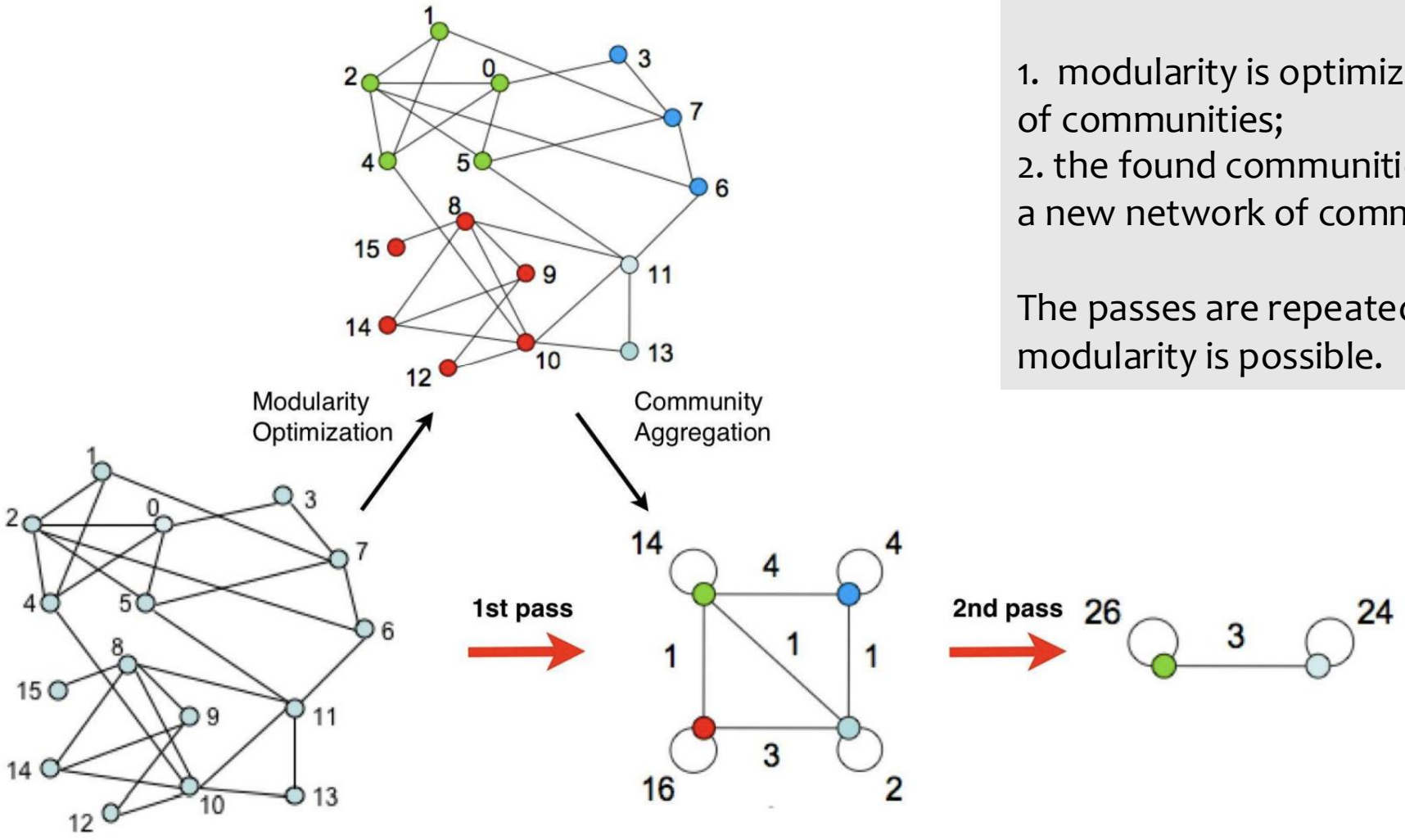
[Vincent et. al: Fast unfolding of communities in large networks, 2018]

Louvain method

The method works iteratively.
Each pass is made of two phases:

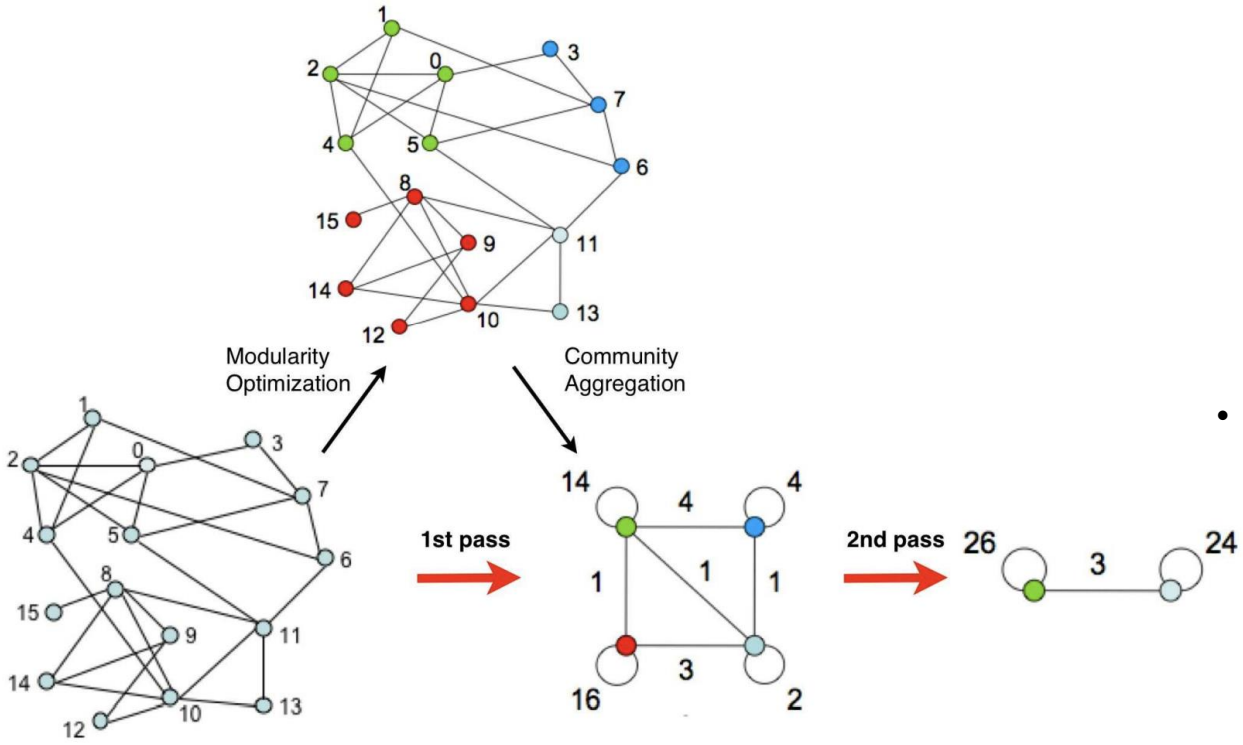
1. modularity is optimized by allowing only local changes of communities;
2. the found communities are aggregated in order to build a new network of communities.

The passes are repeated iteratively until no increase of modularity is possible.



[Vincent et. al: Fast unfolding of communities in large networks, 2018]

Steps of Louvain method



- Start with every node in its community
- Phase1: Modularity optimization
  - Order the nodes and do the following for each node i
    - Move i to the community of neighbor j that leads to maximum ΔQ (gain of modularity)
    - If all ΔQ < 0 (that means no positive gain is possible) then i remains in its current community.
  - Repeatedly process all nodes until ΔQ = 0
- Phase2: Community aggregation
  - Create a weighted network of communities from Phase 1.
    - Nodes = communities from Phase 1
    - Edge weights = sum of edge weights between communities.
    - Edges within a community become 2 self-loops
- Repeatedly apply Phase 1 and 2 until ΔQ = 0

Using networkx for applying Louvain method

```
import community as community_louvain
import matplotlib.cm as cm
import matplotlib.pyplot as plt
import networkx as nx

# load the karate club graph
G = nx.karate_club_graph()

#first compute the best partition
partition = community_louvain.best_partition(G)

# draw the graph
pos = nx.spring_layout(G)
# color the nodes according to their partition
cmap = cm.get_cmap('viridis', max(partition.values()) + 1)
nx.draw_networkx_nodes(G, pos, partition.keys(), node_size=40,
                       cmap=cmap, node_color=list(partition.values()))
nx.draw_networkx_edges(G, pos, alpha=0.5)
plt.show()
```

Prerequisite:

pip install python-louvain

Doc: https://python-louvain.readthedocs.io/en/latest/api.html

Important parameter:

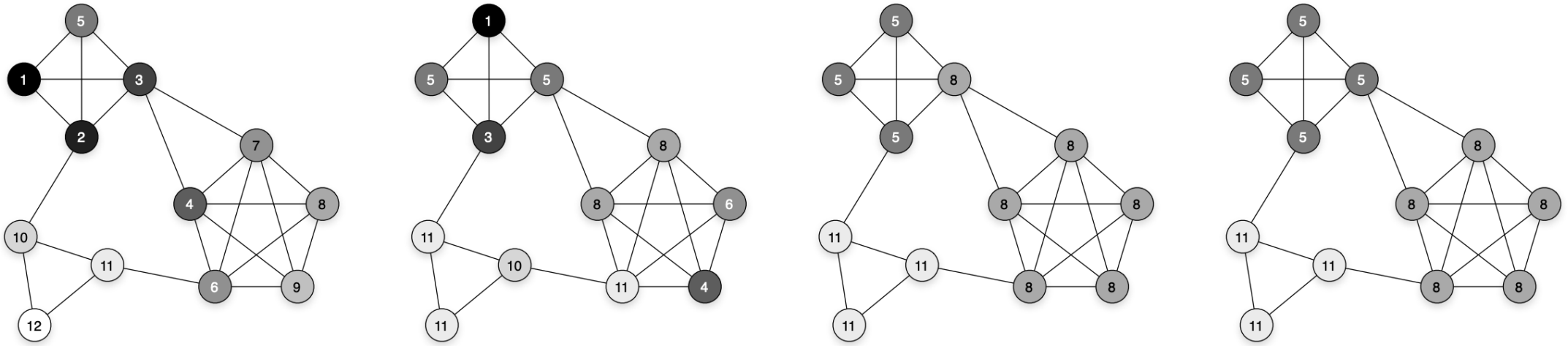
resolution:double, optional

Will change the size of the communities, default to 1.

https://python-louvain.readthedocs.io/en/latest/

Label propagation

It is one of the simplest and fastest clustering methods.
It applies to networks with hundreds of millions of nodes and edges.
How does it work?
Labelling of all nodes in a graph structure.
Labelling starts with random individual labels.
The labels are propagated to neighbour nodes.

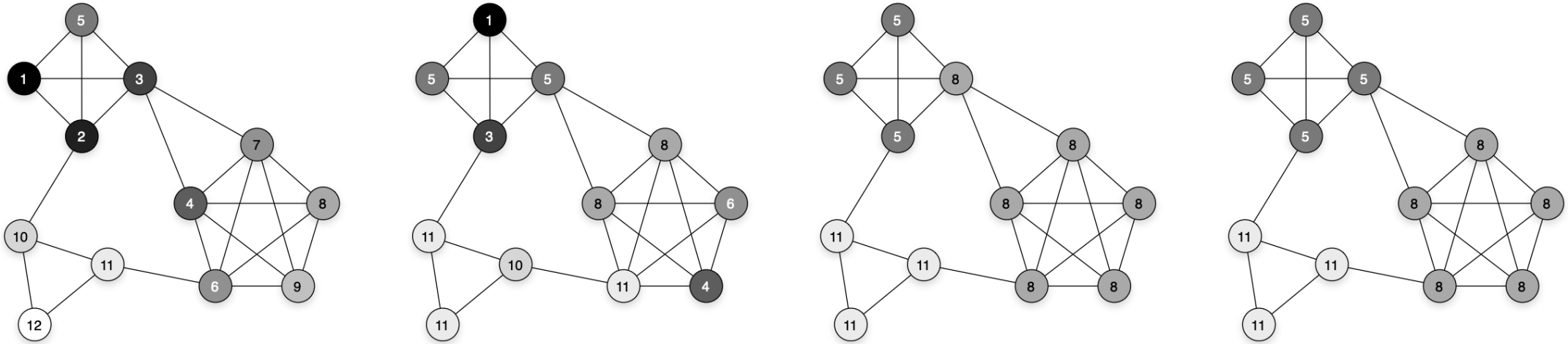


Label propagation in a small network with three communities. The labels and shades of the nodes represent community assignments at different iterations of the label propagation method.
[U. N. Raghavan, R. Albert, and S. Kumara. Near linear time algorithm to detect community structures in large-scale networks.]

Label propagation

Consider a network with n nodes and let Γ_i denote the set of neighbors of node $i \in \{1, \ldots, n\}$. Furthermore, let g_i be the group assignment or community label of node i which we would like to infer.
The label propagation method then proceeds as follows.
Initially, the nodes are put into separate groups by assigning a unique label to each node as gi = i.
Then, the labels are propagated between the nodes until an equilibrium is reached.
At every iteration of label propagation, each node i adopts the label shared by most of its neighbors Γ_i .

$$g_i = \underset{g}{\operatorname{argmax}} \; |\{ j \in \Gamma_i \mid g_j = g\}| .$$



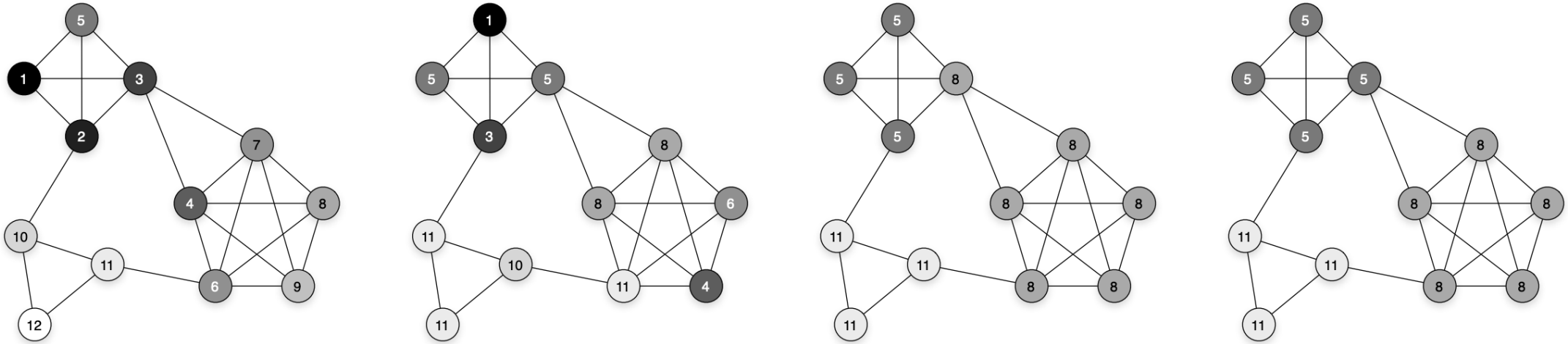
Label propagation in a small network with three communities. The labels and shades of the nodes represent community assignments at different iterations of the label propagation method.

Label propagation

Let A be the adjacency matrix of a network, where A_{ij} is the number of edges between nodes i and j, and A_{ii} is the number of self-edges or loops on node i. The label propagation rule can be written as

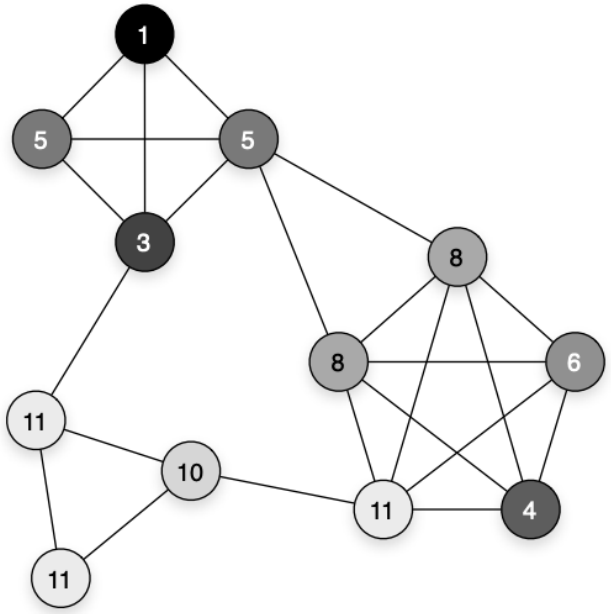
$$g_i = \underset{g}{\operatorname{argmax}} \sum_{j} A_{ij}\, \delta(g_j , g)$$

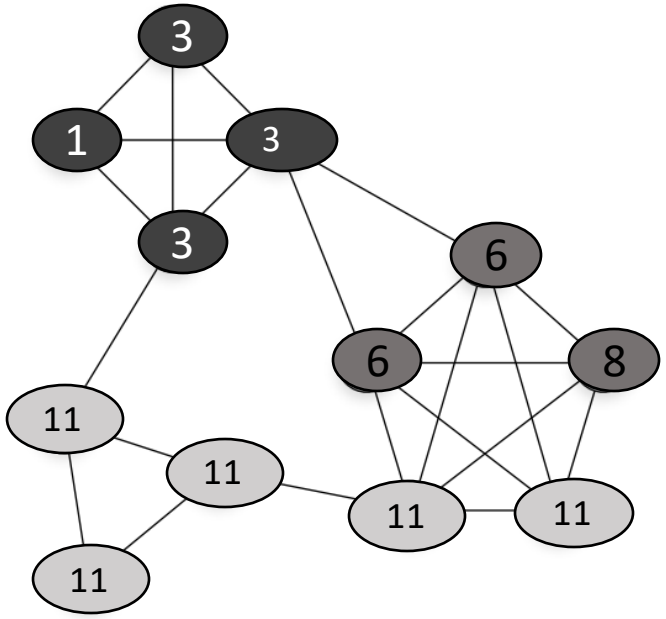
where δ is the Kronecker delta operator that equals one when its arguments are the same and zero otherwise.



Label propagation in a small network with three communities. The labels and shades of the nodes represent community assignments at different iterations of the label propagation method.

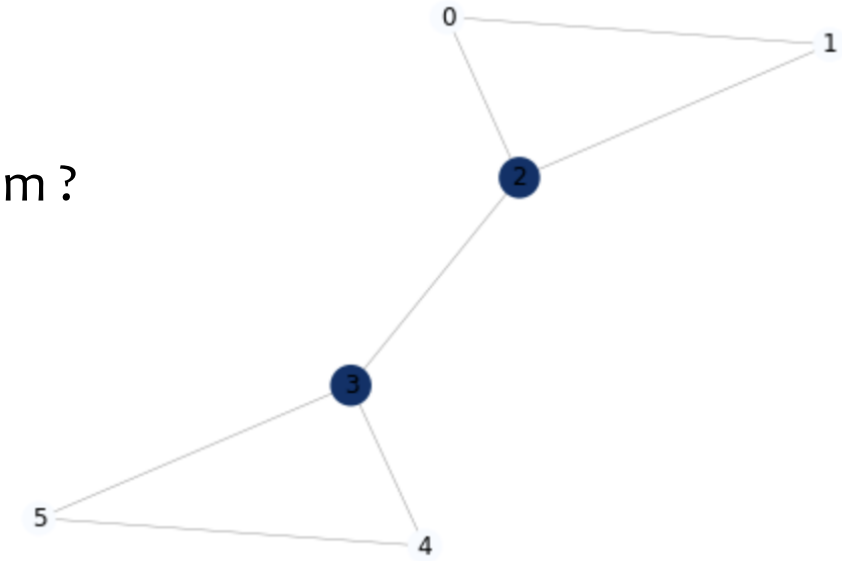
Label propagation (Exercise)





Find all the mistakes from the above label propagation step.
Mark the mistakes with a ✗ sign.

Modularity (Example)



- How to find clusters with networkx's label propagation algorithm ?

import networkx.algorithms.community as nx_comm

G = nx.barbell_graph(3, 0)

nx_comm.modularity(G, nx_comm.label_propagation_communities(G))

0.3571428571428571

[https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.community.quality.modularity.html]

Task 1: **In this assignment you will work with network centrality and modularity using NetworkX library. The details are given below:**

- Load a graph using networkx library from the following csv file:

  'nutrients.csv': https://mitt.uib.no/courses/46883/files/folder/code?preview=5840071

- Measure centrality based on degree/eigenvector centrality using networkx library.
- Detect modularity of the graph using label propagation or louvain method (use networkx libraries).
- Modify the size and color based on the above measures.
- Use spring layout or fruchterman_reingold layout for visualizing the graph.